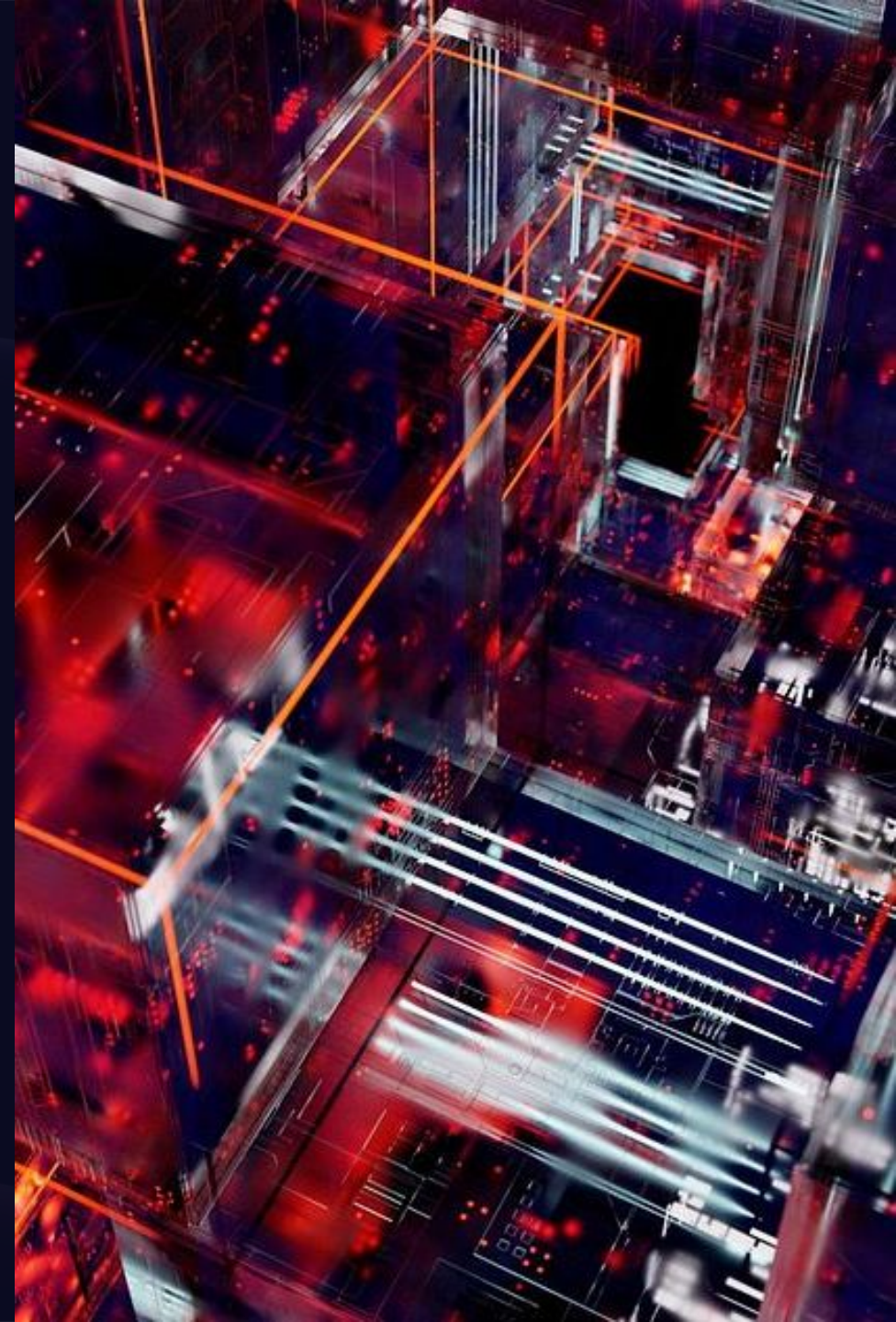


Anatomy of an AI Agent: The Agent Loop

A deep dive into how an LLM becomes a controlled decision-making system — from a single prompt to a full reasoning loop.

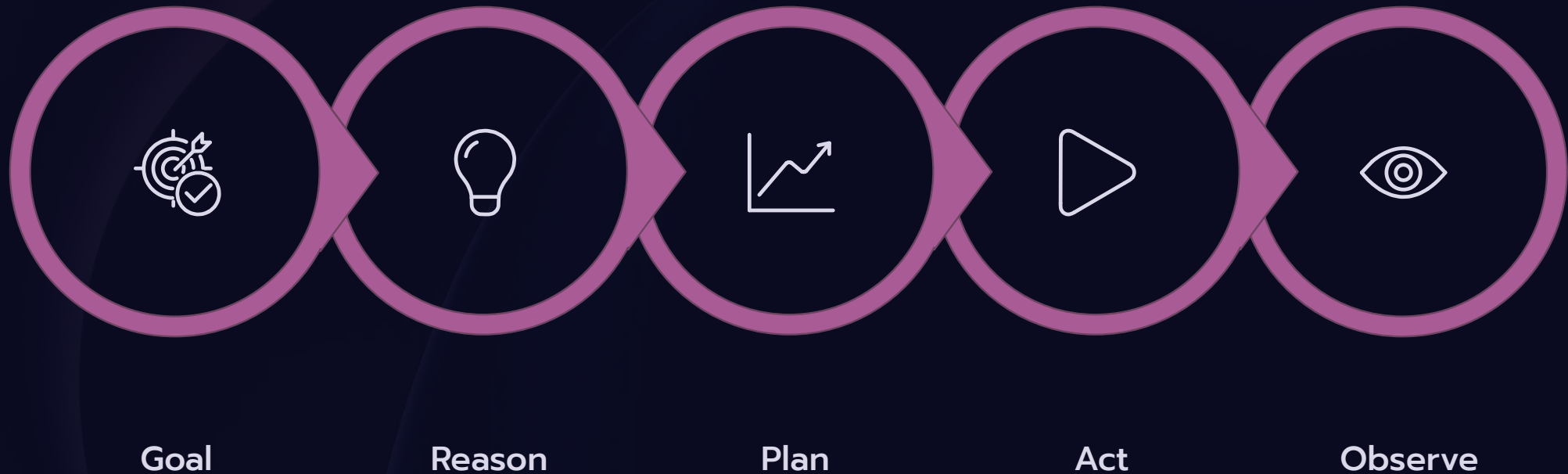
Ramesh Sannareddy



Deck 2 Overview

How an LLM Becomes an Agent

In Deck 1, we established what an agent is. Now we're going to open the box and examine every component inside.



From an LLM call to a controlled decision-making loop



The Simplest Possible LLM Application

Before we build an agent, start with something familiar: a single, stateless LLM call. A user sends a prompt, the LLM generates a response, and the application returns it. There is no loop, no state, no tool use.

```
response = llm.generate(  
    "Create an onboarding checklist for a software engineer."  
)
```

❏ There is **no agent yet**. The application asks the LLM to produce content and returns it — one prompt, one response, done.

Add a Goal

The first and most important distinction between an LLM application and an agent is the nature of the input. A traditional application asks the LLM to *generate something*. An agent gives the LLM an *outcome to accomplish*.

Traditional LLM Application

"Generate an onboarding checklist."

The model produces content. The job is done after one call.

Agent

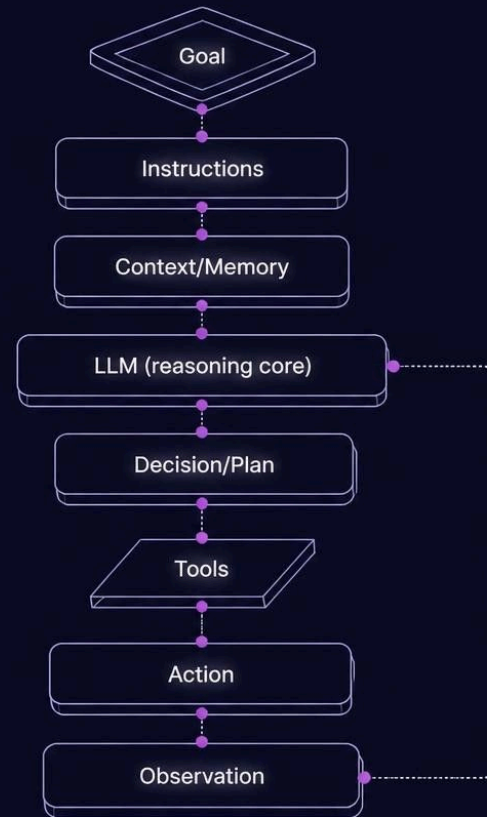
"Onboard employee EMP-1042."

The model must reason, act, observe results, and continue until the outcome is achieved.

An agent starts with an **outcome to accomplish**, not merely a piece of content to generate. This single shift changes the entire software architecture required.

The Anatomy of an Agent

An agent is not just an LLM with a fancy prompt. It is an entire system of components working together. The LLM is the **reasoning component** — the agent is everything built around it.



📌 Emphasize: **The LLM is the reasoning component. The agent is the whole system around it.**

What Does the LLM Actually Do?

Understanding the LLM's role — and its limits — is foundational to safe agent architecture. The LLM is a reasoning engine, not a control plane.

The LLM **Can** Help With

- Interpreting natural language and identifying intent
- Reasoning over available information
- Selecting among available actions
- Generating structured decisions and explanations
- Deciding what information is still needed

The LLM Should **Not** Be Trusted To

- Enforce authorization or validate critical business rules
- Execute arbitrary code or bypass approval
- Guarantee deterministic outcomes

LLM

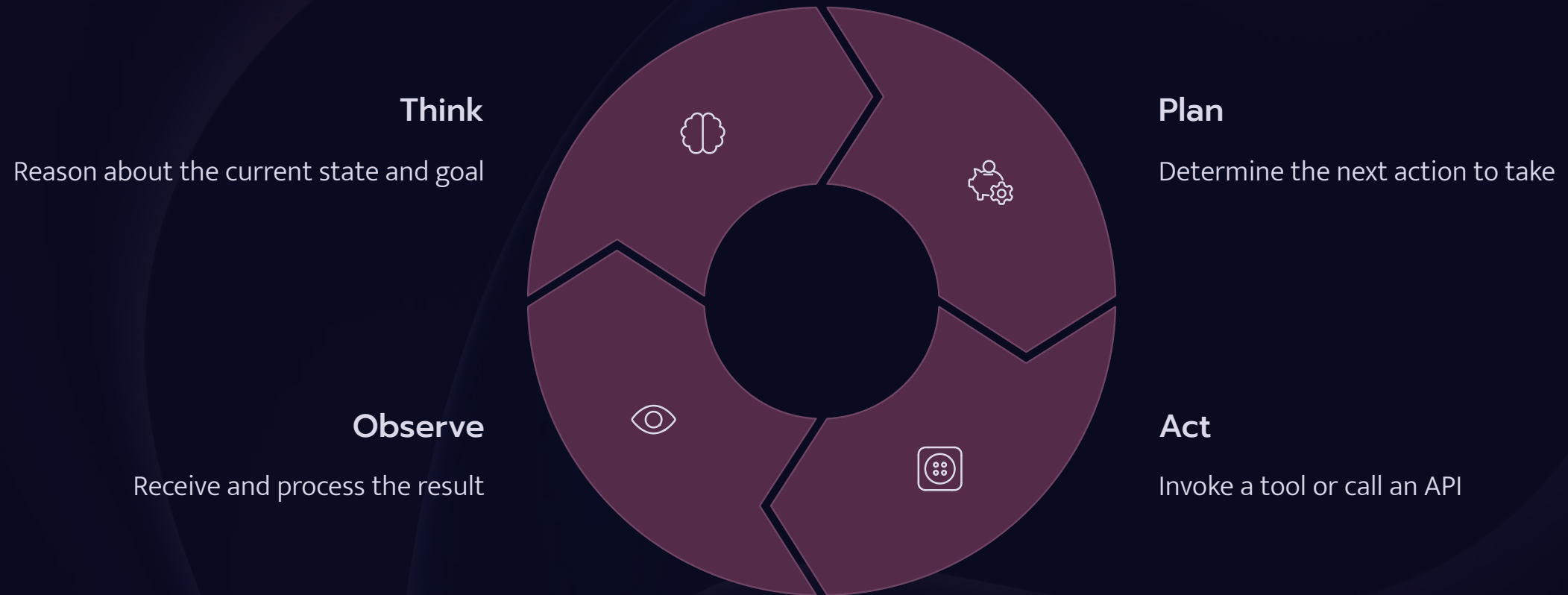
Reasoning capability

Python

Control and execution

The Agent Loop

The loop — not the prompt — is the defining mechanism of an agent. A single LLM call asks a question. A loop pursues a goal across as many iterations as necessary.



Walk Through One Loop: Onboarding EMP-1042

Let's trace a concrete example. Goal: "**Onboard EMP-1042.**" Each iteration incrementally builds the agent's knowledge of the task.

Iteration 1

Think: I need employee information.

Act: `get_employee("EMP-1042")`

Observe: Department = Engineering, Role = Senior Data Engineer

1

2

Iteration 2

Think: I need to determine required access.

Act: `get_required_access(role, department)`

Observe: Access list returned

Iteration 3

Think: I need to check equipment availability.

Act: `check_equipment(role)`

Observe: Equipment status returned

3



The agent is **incrementally building knowledge** about the task — each loop iteration makes it smarter about the current situation.

Planning vs. Execution

Agents separate two distinct cognitive activities: figuring out *what* to do and actually *doing* it. Conflating these leads to brittle systems. Strong agent architectures keep them explicit — and allow the agent to re-plan after observing a result.

Planning Phase

1. Receive the goal
2. Break into ordered steps
3. Determine dependencies between steps
4. Identify required tools for each step

Execution Phase

1. Select the next step
2. Invoke the appropriate tool
3. Receive and validate the result
4. Advance to the next step — or re-plan

Example plan: Get employee → Determine access → Check equipment → Find induction slot → Request approval → Execute. The agent may **re-plan at any step** after observing an unexpected result.

Why the Agent Must Observe

Observation is what separates an agent from a simple script. Without it, the agent blindly executes a plan — with no mechanism to detect or recover from failure.

Without Observation

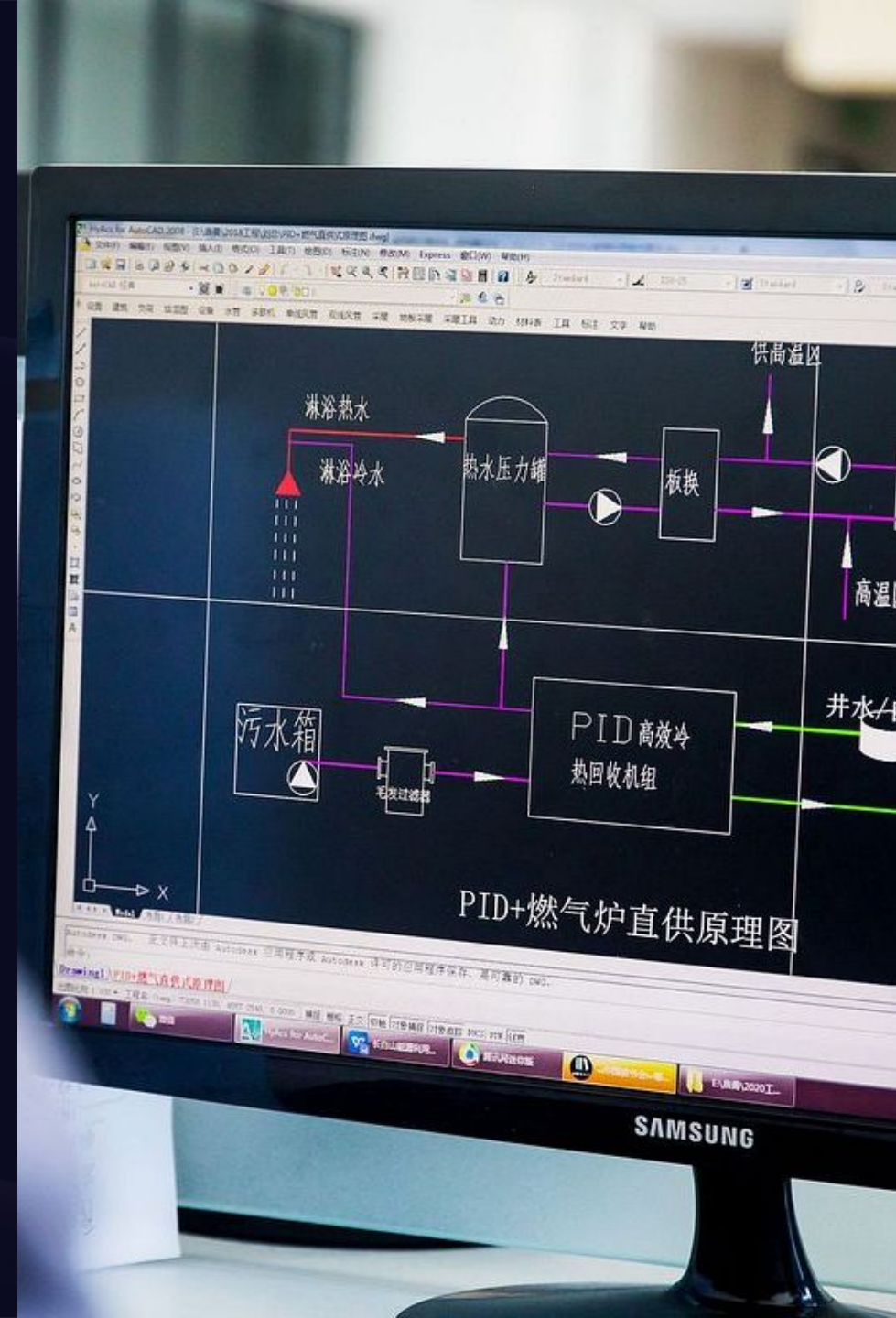
1. Receive goal
2. Generate plan
3. Execute everything

Problem: What if the laptop isn't available? The plan executes anyway — potentially creating inconsistent state.

With Observation

1. Generate plan
2. Check laptop availability
3. Observe: laptop unavailable
4. Re-evaluate the plan
5. Find an alternative → Continue

📄 **Observation closes the loop between intention and reality.** It is the mechanism that makes agents adaptive rather than brittle.



Re-Planning: Adapting to Unexpected Results

A deterministic workflow typically fails or raises an error when a step returns an unexpected result. An agent can reason about what the new information means and adjust its plan accordingly — a fundamentally different operational model.

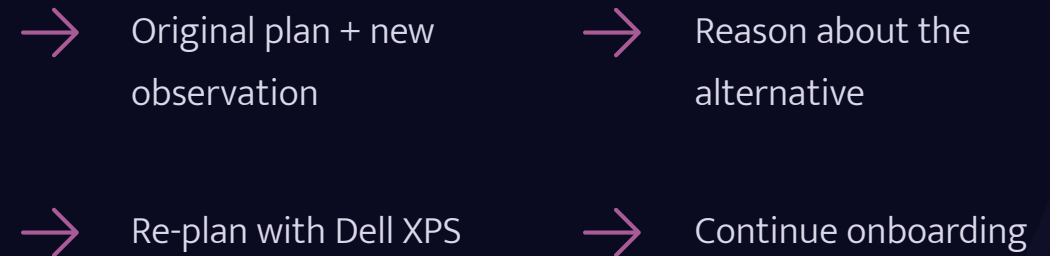
Original Plan

Reserve **MacBook Pro** for EMP-1042.

Tool Returns

MacBook Pro unavailable. Dell XPS available.

Agent Re-Planning Path



This is different from a deterministic workflow where the failure may simply terminate the process and require human intervention to restart.

State: What Does the Agent Know?

There is a critical distinction between **context** (what the LLM can currently see) and **state** (what the application has recorded as having happened). Both are necessary. Neither replaces the other.

Explicit Workflow State

```
{  
  "employee_id": "EMP-1042",  
  "department": "Engineering",  
  "role": "Senior Data Engineer",  
  "equipment": ["laptop", "monitor"],  
  "approval": "pending",  
  "workflow_status": "awaiting_equipment"  
}
```

What State Tracks

- Employee identity and role
- Required access and equipment
- Induction scheduling status
- Approval status
- Overall workflow progress

📌 **Context tells the LLM what it can see. State tells the application what has happened.** This distinction will matter significantly in Lab 3.

Context vs. Memory

Engineers often conflate these two concepts — and that leads to over-engineered solutions. Understand the difference before reaching for a vector database.

Context (Short-Term)

Information currently supplied to the model in a single invocation: the current request, relevant conversation history, current tool results, and current workflow state.

Memory (Long-Term)

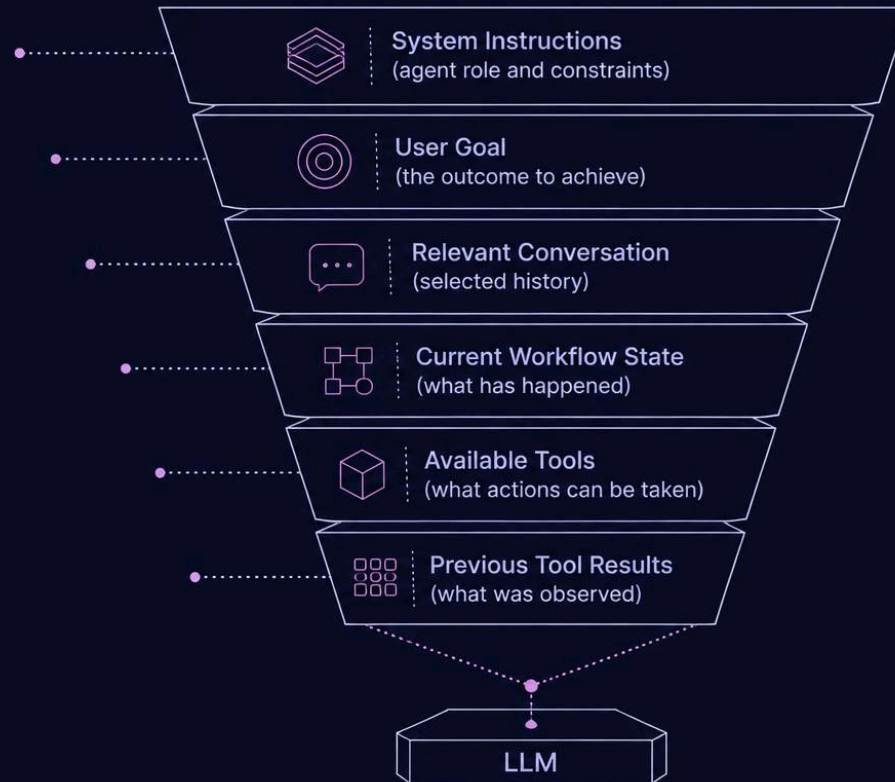
Information retained *beyond* the immediate interaction. Examples: previous interactions, persistent user preferences, prior workflow outcomes. Requires explicit storage and retrieval mechanisms.



Start with **short-term context and workflow state** before introducing sophisticated long-term memory. Don't add a vector database simply because someone says "memory."

What Goes Into the LLM Context?

Context selection is one of the most consequential architectural decisions in an agent system. More context is not automatically better — it increases cost, latency, and can dilute the signal the LLM uses to reason.



Context selection is an architectural decision. Curate carefully — include what the LLM needs to reason well, nothing more.

Structured Decisions

An agent should not rely on free-form natural language for internal decisions. Structured output makes decisions parseable, validatable, observable, testable, and safe to execute — all critical properties in production systems.

Avoid: Free-Form Text

```
"I think we should probably  
check the employee directory  
to get more information."
```

Unparseable. Untestable. Cannot be safely routed to a tool executor.

Prefer: Structured Decision

```
{  
  "action": "get_employee",  
  "arguments": {  
    "employee_id": "EMP-1042"  
  }  
}
```

Deterministic parsing. Validation. Observability. Security.

This structured decision pattern is the bridge into **Lab 2: Tool Calling**, where we implement exactly this mechanism.

Agent Decision vs. Application Control

This is the most critical architectural principle in this deck. The LLM proposes. The application controls. Every action passes through a Python control layer before executing — no exceptions.

01

LLM Decides

"I need employee information." — The LLM reasons and selects an action.

02

Python Validates

"Is `get_employee` an allowed tool? Is `employee_id` present and correctly formatted?"

03

Tool Executes

`get_employee("EMP-1042")` — Only after all validation passes.

04

Result Returned

Department = Engineering. Python returns the structured result to the LLM context.

05

LLM Continues

The LLM reasons about the observation and determines the next action.

Where Things Can Go Wrong

Agentic systems create new failure modes because they create new degrees of freedom. Before building guardrails, you must understand the failure surface.

LLM Failure

Wrong reasoning, hallucinated tool calls, or malformed decisions

Tool Failure

API unavailable, timeout, or unexpected response schema

Data Failure

Employee doesn't exist, incomplete records, stale data

Context Failure

Critical information omitted from the LLM's context window

Loop Failure

Agent enters an infinite loop calling tools without reaching termination

Authorization Failure

Agent attempts a high-consequence action it shouldn't perform

Controlled Autonomy

Autonomy is a spectrum, not a binary. The appropriate level depends on the risk, reversibility, and business impact of each action. Enterprise AI systems must be explicit about where each action sits on this spectrum.



Human Does Everything

No AI involvement in execution



AI Suggests

Draft welcome email → low consequence, instant review



AI Proposes → Human Approves

Reserve laptop → controlled action with human sign-off

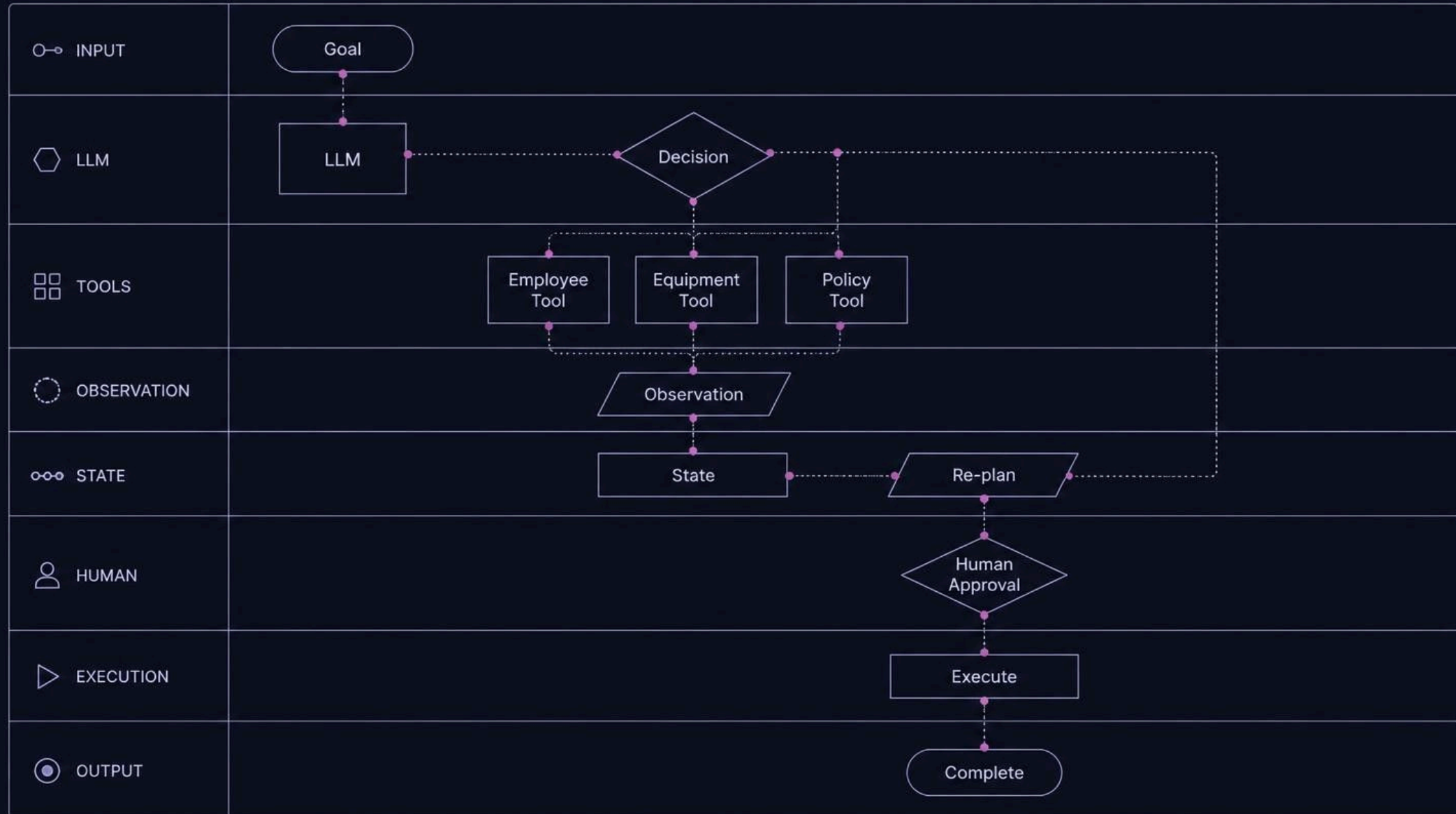


Highly Autonomous + Strong Controls

Terminate employee access → high-consequence, requires robust guardrails and audit trail

The Agent We Are About to Build

Everything we've covered — the loop, state, context, structured decisions, controlled execution — converges in a single reference architecture. We will build this directly in Python, not as a framework abstraction.



Live Demo: The Seed of Everything

Before Lab 1, take five minutes to internalize the canonical agent loop. This tiny structure is the seed from which everything else grows. Every framework abstraction — LangChain, CrewAI, AutoGen — is an elaboration of these four lines.

```
goal = "Onboard EMP-1042"
```

```
while not complete:
```

```
    decision = llm.decide(goal, state, tools)
```

```
    result = execute(decision)
```

```
    state.update(result)
```

- That tiny loop is the architectural mental model. Before you encounter implementation complexity in the lab, hold this structure in mind — everything else is detail layered on top of it.



The Single Most Important Takeaway

An agent is a controlled software loop around an LLM.

Not an LLM with a fancy prompt. A system that can **reason, act, observe, and continue** toward a goal — with your application code in control of every step.



Reasoning

Direction

Information

Decision

Action



LLM Reasons

Interprets, selects, explains



Python Controls

Validates, routes, enforces



The Loop Persists

Until the goal is achieved